CareFlow AI — 30% Milestone Explained

Oct 19, 2026 · @SergeantCooper

1. In short

At 30% complete, CareFlow AI already does the core job end to end for typed messages. A staff member types what the patient says, in English or Hinglish. The system works out which symptoms are present, checks them against 29 medical warning-sign rules, and gives the case a priority from **P1 (see immediately)** to **P5 (can wait)**. It shows the exact words behind every finding and puts the case in a queue sorted by urgency.

This document explains, in simple language:

- what problem we are solving,
- what the 30% version can do today, and how it works inside,
- one patient followed from start to finish,
- how we tested it and what the results were,
- what is left for the remaining 70%, and why we chose this order.

The 30% build lives in the folder `careflow_ai_30_percent/`. The finished system is in `careflow_ai_complete/` and is explained in a separate document.

> **Important rule from the problem statement:** the AI must never diagnose a disease or make the final decision. CareFlow only *sorts and summarises*. A doctor or nurse always decides.

2. The problem in simple words

Triage means deciding which patient should be seen first. In a busy Indian hospital, dozens of people arrive at once. Some describe their problem in Hindi, some in a mix of Hindi and English ("Hinglish"), some in Tamil or Bengali. Some bring printed lab reports, some send a voice note. A nurse has to read or listen to all of it quickly and spot who is in danger.

Things go wrong in three common ways:

1. **Danger signs get missed in the crowd.** A man with chest pain and sweating may wait behind people with a mild cold.
2. **Language gets in the way.** The nurse may not understand the patient's language well.

3. **The same questions get asked again and again,** and nothing is written down in a clear, structured form.

Problem Statement 4 ("CareFlow AI") asks us to build an assistant that:

- takes in text, voice, reports and images in several Indian languages,
- turns them into a clear, structured summary,
- spots possible warning signs and sorts patients by urgency,
- shows where every piece of information came from,
- flags anything missing, unclear, conflicting or suspicious,
- lets a doctor or nurse approve or change everything,
- protects patient data with consent, deletion, role-based access and an activity log.

The AI must **not diagnose**. It is a helper for the people who decide.

3. What we built at 30%

We built the "spine" of the system first. Every later feature (voice, reports, languages, safety checks) plugs into it.

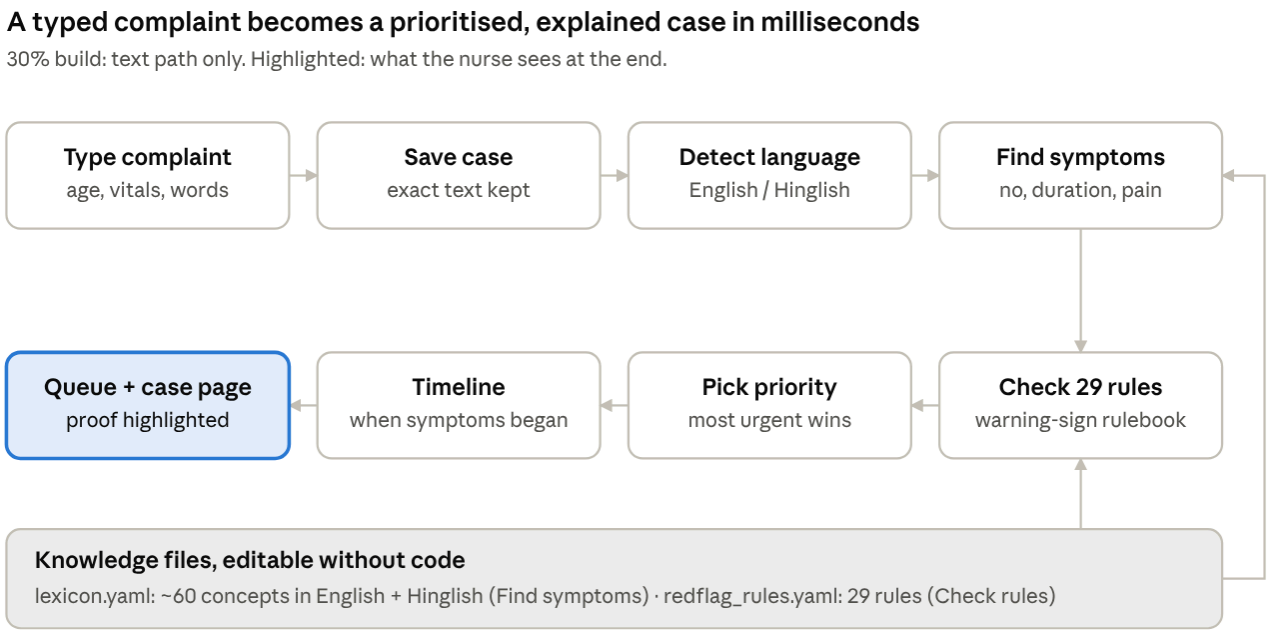
#	Feature	What it means in simple words	Status
1	Web server and database	A small website plus a database file that stores every case, every message and a log of actions	Working
2	Staff login	Doctors, nurses and front-desk staff sign in. Demo users: `dr.mehta`, `nurse.asha`, `intake.ravi`, password `demo`	Working (all roles can do everything for now)
3	New-case form	Staff type the patient's name, age, sex, optional vitals (oxygen level, pulse, BP, temperature) and the complaint	Working
4	Language detection	Tells apart English and Hinglish (e.g. "seene mein dard hai")	Working for these two languages
5	Symptom extraction	Finds about 60 kinds of symptoms and history items, understands "no fever" vs "fever", reads durations ("since 2 days", "1 ghante se") and pain severity ("8/10", "severe")	Working

#	Feature	What it means in simple words	Status
6	Proof for every finding	Each finding remembers the exact words it came from, highlighted in yellow on screen	Working
7	Warning-sign rules	29 rules modelled on standard hospital triage guides (ESI, Manchester Triage, WHO ETAT), e.g. "chest pain + sweating = P1"	Working
8	Priority with a reason	Gives P1-P5 plus one sentence explaining which rule decided it	Working
9	Priority queue	A list of all cases, most urgent first	Working
10	Case page	Shows the priority, the rules that fired, the findings with highlighted proof, and a simple timeline of when each symptom started	Working
11	Activity log	Records logins, case creation, AI results and who viewed which case	Basic version working
12	Tests and evaluation	3 automatic tests, and a script that checks results against our dataset	Working

Not in the 30% build yet: voice notes, reading reports or images, Hindi/Tamil/Bengali, translation, safety checks (fake instructions, conflicts, missing info), follow-up questions, the doctor's note with approval, the patient kiosk with consent, deletion of data, and strict role permissions. Section 10 lists them all.

4. How it works

A case moves through eight simple steps, and the medical knowledge sits in two editable files.



30% build: text to priority, 8 steps

The top row turns words into findings and the bottom row turns findings into a decision aid. The rules never guess: if no rule matches, the case gets a safe default (P4 if any symptom is reported, P5 if none).

5. Worked example: one patient, step by step

The front-desk person types this for a 58-year-old man, Ramesh Kumar. His son speaks for him:

> *"Papa ko 1 ghante se seene mein dard hai, pasina aa raha hai"* (Father has had chest pain for 1 hour, he is sweating)

Here is what CareFlow does with it:

1. **Save.** The case gets a code like `CF-5HEFB`, and the message is stored exactly as typed.
2. **Detect language.** Words like *ko, se, mein, hai* tell the system this is **Hinglish**.
3. **Find symptoms.** The lexicon (our medical word list) matches:
   - "seene mein dard" → **chest pain**
   - "pasina" → **sweating**
4. **Check for "no".** Nothing like *nahi* (not) follows these words, so both are marked **present**.
5. **Read the duration.** "1 ghante se" → **1 hour**. It is attached to both symptoms in the same sentence.

6. **Remember the proof.** Each finding stores its exact character position, e.g. "pasina" = characters 41-47 of the message. The screen highlights exactly those letters.
7. **Apply the rules.** Three rules fire:
   - *Chest pain with associated warning features* (chest pain + sweating) → **P1**
   - *Chest pain in a patient aged 40 or above* → P2
   - *Chest pain* (always needs review) → P3
8. **Pick the priority.** The most urgent level wins: **P1 - Immediate**. Reason shown: "Chest pain with associated warning features (rule RF_CHEST_PAIN_AUTONOMIC)".
9. **Build the timeline.** Chest pain and sweating both started about 1 hour before the case was created.
10. **Queue.** The case jumps to the top of the queue with a red P1 badge. The audit log records "pipeline.completed, priority P1, rules fired ...".

The whole thing takes a few milliseconds. The nurse sees *why* the case is P1 and *which words* caused it, so they can check it in seconds.

6. Tour of the code

The project is small on purpose. It is Python (FastAPI) on the back end and plain HTML/JavaScript on the front end, with no build step and no paid services.

File / folder	What it does
`app/main.py`	The web server. Handles login, creating a case, the queue and the case page. It runs the pipeline as soon as a case is created.
`app/db.py`	The database (one SQLite file). Three tables: **cases**, **sources** (the patient's messages) and **audit** (the activity log).
`app/config.py`	Settings: where the database is, which languages are on, the secret key used for login tokens.
`app/pipeline/orchestrator.py`	The "conductor". Runs each step in order (language, extraction, rules, timeline, summary) and records how long it took.
`app/pipeline/text_utils.py`	Cleans the text (lower-case, Indian digits to 0-9) while remembering original positions, then finds lexicon matches.

File / folder	What it does
`app/pipeline/extract.py`	Turns matches into findings. Handles "no / not / nahi", durations, severity, getting-worse/better, vitals in text (e.g. "SpO2 89"), medicines and allergies. Merges findings from all messages of a case.
`app/pipeline/redflags.py`	The rule engine. Reads the rule file, checks each rule and returns the priority with its reason and proof.
`app/knowledge/lexicon.yaml`	The medical word list: about 60 concepts with English and Hinglish phrases. Editable without touching code.
`app/knowledge/redflag_rules.yaml`	The 29 warning-sign rules in a simple, readable format, each with its priority and the guideline it is based on.
`static/index.html`, `static/js/app.js`	The screens: login, queue, case page, new-case form.
`scripts/evaluate.py`	Runs every supported dataset case through the pipeline, compares with the expected priority, and can load the cases into the app (`--seed`).
`tests/test_core.py`	Automatic checks: priorities, negation, proof positions, the API.

Design choice: medical knowledge (words and rules) is kept in YAML files, not in code. A clinician can read and change the rules without being a programmer, and every rule change is versioned.

7. The dataset

No dataset was provided, and real patient data cannot be used. So we **created our own synthetic dataset** (invented patients, nothing real). It is in `careflow_dataset/` and also zipped as `careflow_dataset.zip`.

- **45 patient cases**, each with the input (text, voice note or report) and the "correct answer" we expect: priority, warning-sign rules, flags and symptoms.
- **5 languages:** English, Hindi, Hinglish, Tamil, Bengali.
- **7 lab reports** (PDFs and photos) and **4 voice notes**.

- **20 fake-instruction attacks** and **20 harmless look-alike sentences**, used to test the safety checks.
- A **generator script** that rebuilds everything from scratch.

Which part the 30% build uses: only the cases that are purely typed text in English or Hinglish. That is **25 of the 45 cases**, spread across all five priority levels. The other 20 need voice, reports or other languages, which arrive in later milestones.

8. Testing and results

The system gave the expected priority for **24 of the 25** supported cases, and never rated a case more urgent than it should be.

Check	Result
Priority exactly right	24 / 25 cases (96%)
Cases rated less urgent than they should be	1 (CF-013)
Cases rated more urgent than they should be	0
Automatic tests (`pytest`)	3 / 3 pass
Time to process one typed case	a few milliseconds

The one miss, explained: case CF-013 says *"It feels like an elephant is sitting on me"*. A doctor reads that as chest pain, but our word list does not contain the phrase, so the rules miss it. This is the weakness of keyword systems, and the reason the complete version adds an optional small AI language model that understands indirect descriptions. Even then, the model is only allowed to *add* a finding when it quotes the patient's exact words.

What the automatic tests check:

- Six sample messages get the right priority, from P1 down to P5, including a Hinglish one and one with low oxygen.
- "I have a cough but no fever" gives cough = yes and fever = no.
- The saved proof position points to exactly the right words.
- The website: login is required, a stroke case created through the API becomes P1 and appears first in the queue.

Honest note: our team wrote both the expected answers and the rules, so this score shows that the system is *consistent*. It is not medical proof. Real validation would need cases labelled by doctors.

9. How to run and demo it

You need a Linux or Mac computer with Python 3.11 or newer. No GPU or internet service is needed after installing.

```
cd careflow_ai_30_percent
./run.sh
```

The first run installs the packages, loads the 25 supported dataset cases and starts the website at **http://localhost:8001**. Sign in as `dr.mehta`, password `demo`.

A 3-minute demo for the organisers:

1. **Queue.** Show the list sorted by urgency: red P1 cases (stroke signs, anaphylaxis, suicidal thoughts, very low BP) are at the top.
2. **Open a P1 case**, e.g. Priya Menon (lips and throat swelling after peanuts). Show the rule that fired and the yellow highlighted words that triggered it.
3. **Create a new case** with *New case*:
   - Type *"4 din se tez bukhar, sar dard"* (high fever for 4 days, headache).
   - The system picks **P3**, with the reason "Fever for 3 days or more".
   - Change it to *"no fever, mild cough"* to show that "no" is understood.
4. **Explain the plan:** these steps are the spine. Voice, reports, more languages and safety checks plug into it next.

Other commands:

- `.venv/bin/python -m pytest -q`: run the tests
- `.venv/bin/python -m scripts.evaluate`: print the result for every supported dataset case

10. What is left (the other 70%) and why this order

Why we built the spine first. Every other feature feeds into, or reads from, the same three things: the *findings with proof*, the *rules* and the *queue*. A voice note just becomes more text. A lab report just adds more findings. A safety check just adds flags. Getting this core right and tested first means each later piece is a plug-in, not a rewrite. The three core engine files (`text_utils.py`, `extract.py`, `redflags.py`) were carried into the finished version **without changes**.

Milestone	What gets added	Why it matters for the problem statement
Multimodal	Voice notes (speech-to-text with a confidence score), reading lab reports and photos (OCR), lab values with normal ranges	"Accept symptoms through text or voice, extract information from medical reports using OCR"
Multilingual	Hindi, Tamil, Bengali words; English translation of each message with a confidence score	"Understand and translate English, Hindi, regional languages … indicate when unsure"
Safety	Detecting fake instructions in inputs, conflicts between sources, unreadable reports, hidden text in PDFs, missing information	"Flag these issues rather than guess or silently correct them"
Clinician workflow	Follow-up questions in the patient's language, a draft note the doctor edits and signs, changing priority with a reason, live alerts, longer waits moving up	"Allow reviewers to approve, edit, or override"; "suggest follow-up questions"
Privacy	Patient kiosk with consent, withdraw/delete data, strict role permissions, tamper-proof activity log	"Ask for consent, allow data to be withdrawn or deleted, restrict by role, keep a record"
Bonus	Optional small language model (under 20B parameters) with strict guardrails; FHIR export; analytics	Bonus points: SLM use and extra innovation

The full checklist is in `careflow_ai_30_percent/TODO.md`. All of these items are finished in `careflow_ai_complete/`, which the second document explains.

11. Questions the organisers may ask

Q: Is this a large AI model making medical decisions? No. The 30% build uses no large language model at all. It uses a medical word list and clear rules that anyone can read. It only suggests a review order. A doctor or nurse decides.

Q: Why rules instead of "just use ChatGPT"? Rules are predictable, explainable and fast, and they cannot be tricked by clever wording. For safety-critical sorting, we want every P1 to be explainable as "this rule, because of these words". An AI model is added later only as a helper, and it can only *raise* urgency, never lower it.

Q: How do I know where a finding came from? Every finding stores the exact position of the words it came from, and the screen highlights them in yellow. Nothing appears without proof.

Q: What does "P1-P5" mean? P1 Immediate, P2 Emergent, P3 Urgent, P4 Less urgent, P5 Non-urgent. The scale is modelled on the Emergency Severity Index used in many hospitals.

Q: What happens with a message the system doesn't understand? If no symptoms are found, the case is still saved and shown in the queue. Any symptom without a warning sign gets P4, so it is reviewed rather than ignored. In the complete version, missing or unclear information is also flagged with follow-up questions.

Q: Is patient data safe? In the 30% build, data stays in a local database on the hospital's computer, and every action is logged. Consent, deletion and strict per-role access come in the next milestones and are finished in the complete version.

Q: Where did the data come from? We created it ourselves. All 45 cases are synthetic, so no real patient data was used.